

```
performTransactions :: (Transaction, Transaction, Transaction) -> Int -> Maybe Int
performTransactions (t1, t2, t3) balance = case updateBalance t1 balance of
  Nothing -> Nothing
  Just balance1 -> case updateBalance t2 balance1 of
    Nothing -> Nothing
    Just balance2 -> updateBalance t3 balance2
```

```
performTransactions :: (Transaction, Transaction, Transaction) -> Int -> Maybe Int
performTransactions (t1, t2, t3) balance = case updateBalance t1 balance of
  Nothing -> Nothing
  Just balance1 -> case updateBalance t2 balance1 of
    Nothing -> Nothing
    Just balance2 -> updateBalance t3 balance2
```

```
instance Monad Maybe where
  (>=) :: Maybe a -> (a -> Maybe b) -> Maybe b
  Nothing >= _ = Nothing
  Just x >= f = f x

return :: a -> Maybe a
return = Just
```

```
performTransactions :: (Transaction, Transaction, Transaction) -> Int -> Maybe Int
performTransactions (t1, t2, t3) balance = case updateBalance t1 balance of
  Nothing -> Nothing
  Just balance1 -> case updateBalance t2 balance1 of
    Nothing -> Nothing
    Just balance2 -> updateBalance t3 balance2
```

```
instance Monad Maybe where
  (>>=) :: Maybe a -> (a -> Maybe b) -> Maybe b
  Nothing >>= _ = Nothing
  Just x >>= f = f x

  return :: a -> Maybe a
  return = Just
```

```
performTransactions :: (Transaction, Transaction, Transaction) -> Int -> Maybe Int
performTransactions (t1, t2, t3) balance =
  updateBalance t1 balance >>= updateBalance t2 >>= updateBalance t3
```

```
canReachIn3 :: Position -> Position -> Bool
```

```
canReachIn3 start end = end `elem` thirdMove
```

```
  where
```

```
    firstMove = moveKnight start
```

```
    secondMove = concatMap moveKnight firstMove
```

```
    thirdMove = concatMap moveKnight secondMove
```

```
canReachIn3 :: Position -> Position -> Bool
canReachIn3 start end = end `elem` thirdMove
```

```
  where
```

```
    firstMove = moveKnight start
```

```
    secondMove = concatMap moveKnight firstMove
```

```
    thirdMove = concatMap moveKnight secondMove
```

```
instance Monad [] where
```

```
  (>=>) :: [a] -> (a -> [b]) -> [b]
```

```
  xs >=> f = concatMap f xs
```

```
  return :: a -> [a]
```

```
  return x = [x]
```

```
canReachIn3 :: Position -> Position -> Bool
canReachIn3 start end = end `elem` thirdMove
  where
    firstMove = moveKnight start
    secondMove = concatMap moveKnight firstMove
    thirdMove = concatMap moveKnight secondMove
```

```
instance Monad [] where
  (>=>) :: [a] -> (a -> [b]) -> [b]
  xs >=> f = concatMap f xs

  return :: a -> [a]
  return x = [x]
```

```
canReachIn3 :: Position -> Position -> Bool
canReachIn3 start end =
  end `elem` (moveKnight start >=> moveKnight >=> moveKnight)
```

```
doubleSquareHalveAndIncrementL :: Double -> Writer [String] Double
doubleSquareHalveAndIncrementL x = Logged (finalResult, log1 ++ log2 ++ log3)
  where
    Logged (result1, log1) = doubleL x
    Logged (result2, log2) = squareL result1
    Logged (result3, log3) = halveL result2
    finalResult = result3 + 1
```

```
doubleSquareHalveAndIncrementL :: Double -> Writer [String] Double
doubleSquareHalveAndIncrementL x = Logged (finalResult, log1 ++ log2 ++ log3)
  where
    Logged (result1, log1) = doubleL x
    Logged (result2, log2) = squareL result1
    Logged (result3, log3) = halveL result2
    finalResult = result3 + 1
```

```
instance Monad (Writer a) where
  (>>=) :: Writer l a -> (a -> Writer l a) -> Writer l a
  (Writer (x, log1)) >>= f = Writer (y, log1 ++ log2)
    where
      Writer (y, log2) = f x

  return :: a -> Writer l a
  return x = Writer (x, [])
```



```
doubleSquareHalveAndIncrementL :: Double -> Writer [String] Double
doubleSquareHalveAndIncrementL x = Logged (finalResult, log1 ++ log2 ++ log3)
  where
    Logged (result1, log1) = doubleL x
    Logged (result2, log2) = squareL result1
    Logged (result3, log3) = halveL result2
    finalResult = result3 + 1
```

```
instance Monad (Writer a) where
  (>>=) :: Writer l a -> (a -> Writer l a) -> Writer l a
  (Writer (x, log1)) >>= f = Writer (y, log1 ++ log2)
    where
      Writer (y, log2) = f x

  return :: a -> Writer l a
  return x = Writer (x, [])
```

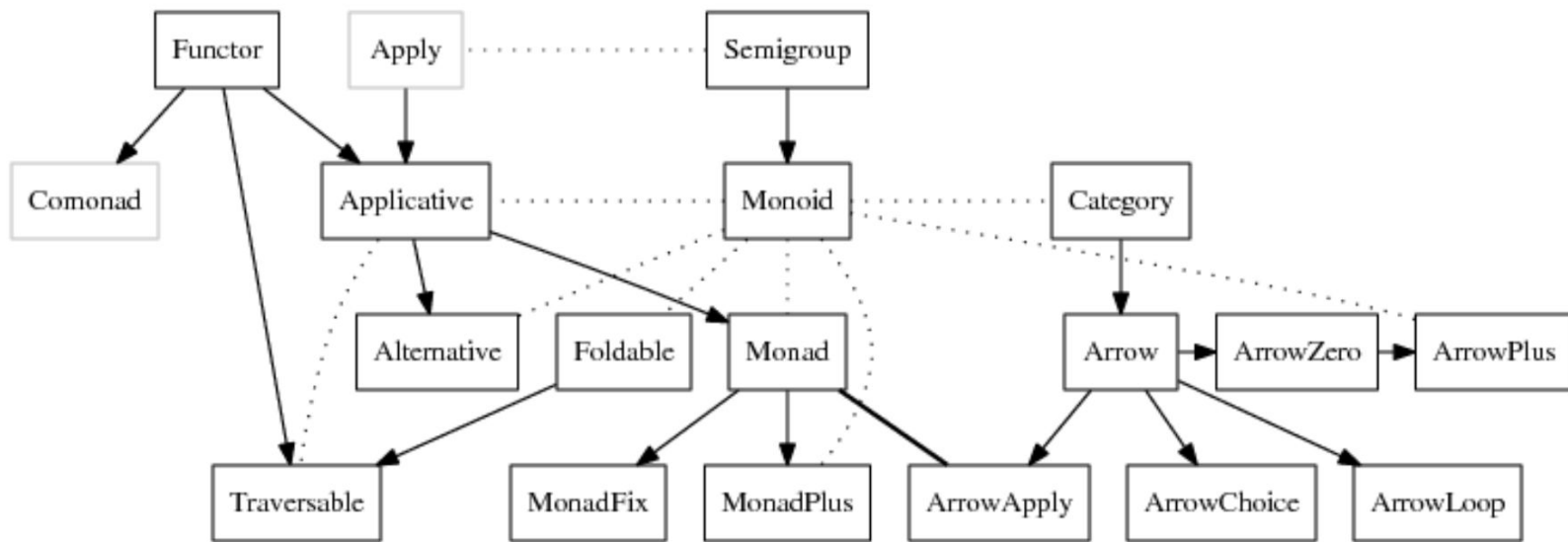
```
doubleSquareHalveAndIncrementL' :: Double -> Writer [String] Double
doubleSquareHalveAndIncrementL' x = doubleL x >>= squareL >>= halveL >>= return . (+ 1)
```

```
performTransactions :: (Transaction, Transaction, Transaction) -> Int -> Maybe Int
performTransactions (t1, t2, t3) balance =
    updateBalance t1 balance >=> updateBalance t2 >=> updateBalance t3
```

```
canReachIn3 :: Position -> Position -> Bool
canReachIn3 start end =
    end `elem` (moveKnight start >=> moveKnight >=> moveKnight)
```

```
doubleSquareHalveAndIncrementL' :: Double -> Writer [String] Double
doubleSquareHalveAndIncrementL' x = doubleL x >=> squareL >=> halveL >=> return . (+ 1)
```

<https://philipnilsson.github.io/Badness10k/escaping-hell-with-monads/>



- Solid arrows point from the general to the specific; that is, if there is an arrow from `Foo` to `Bar` it means that every `Bar` is (or should be, or can be made into) a `Foo`.
- Dotted lines indicate some other sort of relationship.
- `Monad` and `ArrowApply` are equivalent.
- `Apply` and `Comonad` are greyed out since they are not actually (yet?) in the standard Haskell libraries *.